

RUPTURE

Rapport technique final — V5 (finalisation complète)

Nature de ce document	Rapport technique final et exhaustif du premier cycle complet de RUPTURE : concept → prototype → développement → équilibrage → bêta réelle → monétisation → sécurisation → préparation stores → documentation. Conçu pour permettre de reprendre le projet des mois plus tard sans reconstituer son historique.
Date du rapport	13 septembre 2026
Version finale	V5 — « build bêta finale »
Commit réellement déployé	bc3b8637c0384dc43d2d8e0b45c148f3375fc5f9 (bc3b863), branche main, vérifié via l'API GitHub Actions (run 34776944514, conclusion success).
URL permanente (bêta)	https://scuillerjack-oss.github.io/rupture-v0/ — inchangée depuis V0, jamais cassée par cette passe de finalisation.
Dépôt	github.com/scuillerjack-oss/rupture-v0 , branche main.

Convention de couleur utilisée dans tout ce document : **vert (RÉEL)** — fonctionne réellement, vérifié. **violet (SIMULÉ)** — code réel et testé, mais qui ne parle à aucune plateforme externe réelle ; clairement étiqueté « test/simulation » dans l'interface elle-même. **orange (À FAIRE / LIMITE CONNUE)** — préparé mais non exécuté, ou limitation assumée. **rouge (NON TESTABLE ICI)** — nécessite un compte, un service ou un environnement absent de ce contexte d'exécution distant.

1. Résumé du cycle complet

RUPTURE est un prototype de stratégie/simulation de crise : le joueur incarne une Anomalie qui se propage sur un réseau de 18 territoires face à une Humanité qui détecte, mobilise et tente de la maîtriser — une course symétrique à 100% des deux côtés. Développé en JavaScript vanilla (ES modules, sans framework UI), bundlé avec Vite, déployé en PWA sur GitHub Pages avec déploiement automatique (GitHub Actions) à chaque push sur main.

Étapes déjà parcourues avant cette passe de finalisation

- **V0-V2** : prototype jouable, mini-tutoriel, difficulté, sauvegarde locale avec migrations, PWA installable, première batterie de simulations automatisées (non-régression).
- **V3** : nouvelle branche Dangerosité, échelle Région → Zone → Monde, recalibrage de durée de partie.
- **V4** : correction du lien Conscience → Réponse mondiale, recalibrage de la hiérarchie de difficulté, effet mesuré de la Discrétion sur la mobilisation.
- **V4.1** : audio procédural (premher jet, drone continu), récapitulatif de fin de partie.
- **V4.2** : correction d'un bug de musique inaudible sur haut-parleur de téléphone réel (registre trop grave), investigation rigoureuse (4 mécanismes testés sur 90 parties chacun) pour renforcer la Résilience face à la Réponse mondiale — **aucun changement retenu**, conformément à la consigne explicite de ne jamais forcer une idée qui dégrade le jeu quand aucun levier sûr n'a été trouvé.
- **Audit économique** (docs/RUPTURE_Audit_Economique_Precommercialisation.pdf) : modèle de monétisation étudié et validé en amont de cette passe — source de vérité utilisée telle quelle ici, aucun paramètre réinventé.

Cette passe V5 est explicitement une **finalisation**, pas une nouvelle refonte : aucun changement de gameplay, d'équilibrage ou d'architecture fonctionnelle n'a été fait au-delà de ce qui est décrit dans ce document. Le résultat de la bêta manuelle réelle est accepté tel quel.

2. Résultats de la bêta réelle et correctifs appliqués

La bêta manuelle réelle (téléphone Android, PWA installée depuis l'URL permanente) a remonté deux défauts précis, tous deux corrigés dans cette passe.

2.1 Reprise automatique de partie au relancement (corrigé)

Défaut : fermer complètement l'application puis la rouvrir reprenait directement la partie en cours, sans jamais repasser par le menu — le bouton « Reprendre la partie » existait déjà mais n'était en pratique jamais utile.

Cause : `src/main.js` chargeait la sauvegarde directement au démarrage (`loadState()` | `createInitialState()`). Une fermeture complète et une éviction silencieuse de la `WebView` par l'OS mobile en arrière-plan sont strictement indiscernables du point de vue de la page : les deux se traduisent par un rechargement complet du module.

Correctif : le démarrage part désormais toujours du menu (`state = createInitialState()`). Le bouton « Reprendre la partie » préexistant devient le seul chemin de reprise explicite. Le cycle premier-plan/arrière-plan pendant une session normale ouverte (`document.hidden`, sans rechargement de page) n'est pas affecté — la partie continue de se mettre en pause/reprendre normalement pendant une session en cours.

Vérifié par : suite Playwright dédiée (voir §7) reproduisant fidèlement un cycle fermeture complète → réouverture (nouvel onglet dans le même contexte navigateur, donc même `localStorage`) : confirme un retour au menu sans reprise automatique, et que le bouton « Reprendre la partie » restaure bien la partie sauvegardée quand il est cliqué explicitement.

2.2 Bande-son perçue comme « une seule note continue » (corrigé)

Défaut : la musique procédurale de V4.1/V4.2 (trois oscillateurs jouant en continu pendant toute la partie) a été perçue comme fatigante, une tonalité unique qui ne varie jamais assez.

Correctif : remplacement complet du moteur musical (`src/audio/audio.js`) par une ambiance événementielle espacée — **aucun son ne joue en continu**. Une note (parfois deux, 1 fois sur 5, à quelques centaines de ms d'écart) toutes les 15 à 45 secondes environ, avec un silence occasionnellement doublé (jusqu'à ~75s). Chaque événement varie : note choisie au hasard dans un petit accord mineur cohérent (165-440 Hz, registre déjà validé audible sur haut-parleur de téléphone en V4.2), timbre (sinus/triangle), durée (1,4 à 3,4s) et volume légèrement aléatoires. Effets sonores (achat, victoire, défaite, changement de phase) non modifiés — déjà validés en bêta.

Conforme à l'instruction explicite reçue : « le silence partiel est largement préférable à une mauvaise bande-son continue » — aucune tentative de retravailler un système continu qui risquait de rester fatigant sous une autre forme.

Vérifié par : un test Playwright dédié utilisant l'horloge virtuelle du navigateur (`page.clock`) pour avancer 6 minutes de temps de jeu en quelques secondes de test réel, en interceptant chaque création d'oscillateur audio. Résultat mesuré : **12 événements sonores distincts sur 6 minutes virtuelles, avec des silences réels de 16 à 42 secondes entre chaque** — confirmant à la fois l'absence de son continu et l'absence de silence permanent (musique reste présente, juste espacée). Voir §7.2 pour le détail des mesures.

3. Modèle économique intégré (source de vérité : l'audit déjà validé)

Aucun paramètre n'a été réinventé. Les valeurs ci-dessous proviennent telles quelles de docs/RUPTURE_Audit_Economique_Precommercialisation.pdf, déjà validé avant cette passe et désormais committé dans le dépôt comme référence permanente.

Paramètre	Valeur retenue
Premium	Achat unique, 2,99€. Retire la publicité + débloque la vitesse ×4. Aucun avantage stratégique, aucune méta-monnaie, aucune dégradation du contenu gratuit (×1/×2, toutes les orientations restent jouables sans Premium).
Publicité	Interstitialle AdMob uniquement (pas de bannière, pas de vidéo récompensée dans le modèle retenu). Jamais pendant une partie.
Fréquence	Jamais autour de la toute première partie réellement terminée. Ensuite : 1 publicité par partie si sa durée réelle (horloge murale) est ≥ 3 minutes ; sinon, 1 publicité toutes les 2 parties courtes (taux dégradé pour ne pas punir les parties rapides).
Plateforme	Google Play / Android en priorité. iOS hors périmètre de l'audit économique (non traité en détail dans cette passe non plus).

Ce modèle a été transformé en architecture logicielle exploitable (§4/§5), pas seulement documenté : la règle de fréquence ci-dessus est un algorithme réel, testé unitairement (§7), et non un exemple de code laissé de côté.

4. Comment fonctionne la publicité (architecture réelle)

Fichier : `src/services/ads.js`. Instancié comme les autres services (`src/services/index.js`), branché dans `main.js` au moment où une nouvelle partie est demandée après une partie précédemment terminée.

Fonction	Statut	Détail
<code>shouldShowInterstitial(context)</code>	RÉEL	Fonction pure, testée (7 tests dédiés), applique exactement la règle de fréquence de l'audit économique. Ne dépend d'aucun SDK.
<code>showInterstitialAd()</code>	SIMULÉ	Aucun SDK connecté. En dev/bêta : affiche un écran overlay clairement étiqueté « simulation », avec un bouton « Continuer » explicite. En environnement 'commercial' : refuse d'agir (<code>resultKind: 'not-connected'</code>) plutôt que de simuler un affichage qui laisserait croire à un vrai revenu publicitaire.
<code>createAdToken()</code> / <code>consumeAdToken()</code>	PRÉPARÉ, non utilisé	Jetons anti-rejeu à usage unique. Aucune mécanique de publicité à récompense n'existe dans le modèle économique retenu (l'audit ne mentionne qu'un exemple hypothétique) — cette structure existe pour accueillir une telle mécanique plus tard sans réécriture, sans en inventer une aujourd'hui.
<code>isRewardedAdAvailable()</code> / <code>showRewardedAd()</code>	NON ACTIF	Renvoient explicitement « indisponible » — documente honnêtement qu'aucune récompense publicitaire n'existe actuellement, plutôt que de laisser un flou.

Consentement publicitaire (RGPD/EEE) : à intégrer avant le premier appel publicitaire réel si l'audience européenne est ciblée (SDK gratuit Google User Messaging Platform) — non fait ici, documenté au §9.3.

5. Comment fonctionne Premium (architecture réelle)

Fichier : `src/services/premium.js`. Séparation stricte entre `isPremium()` (cache d'affichage rapide) et `getEntitlementSource()` (d'où vient cette valeur) — pensée dès maintenant pour qu'une future vérification Play Billing n'ait qu'à écrire 'store-verified' au bon endroit, sans réécrire le reste du jeu.

Fonction	Statut	Détail
<code>isPremium()</code> / <code>getEntitlementSource()</code>	RÉEL	Lisent un état réel (localStorage), pas un exemple de code. Les sources possibles : 'simulated-test' (dev/bêta), 'store-verified' (réservé à une future intégration réelle — jamais écrit par le code actuel), ou null si absente.
<code>getAvailableSpeeds()</code> (config/runtime.js)	RÉEL, mais inactif en bêta	Restreint réellement la vitesse $\times 4$ aux joueurs Premium — mais seulement si <code>env === 'commercial'</code> . La PWA bêta déployée reste $\times 1/\times 2/\times 4$ intégralement libres pour tous, conformément à la consigne de ne jamais restreindre la bêta en cours.
<code>purchasePremium()</code> / <code>restorePurchases()</code>	SIMULÉ	Aucun compte Google Play Billing/App Store connecté. En dev/bêta : accordent un entitlement marqué 'simulated-test' après un court délai simulé — jamais confondu avec un achat réel. En environnement 'commercial' : refusent explicitement d'agir plutôt que d'accorder Premium gratuitement.

Testé explicitement : `purchasePremium()` et `restorePurchases()` ne peuvent PAS accorder Premium gratuitement en build 'commercial' — un test dédié vérifie ce refus (§7).

6. Sécurisation raisonnable de la monétisation

Objectif explicitement borné : une sécurité raisonnable pour un premier jeu commercial — simple, maintenable, économique, suffisamment sérieuse. Aucune infrastructure disproportionnée n'a été construite.

6.1 Ce qui est réellement en place aujourd'hui

- **Séparation source d'entitlement** : un Premium actif est toujours accompagné d'une source traçable ('simulated-test' ou 'store-verified'), jamais une simple valeur booléenne opaque.
- **Jetons anti-rejeu** pour un futur événement publicitaire à récompense : un jeton consommé ne peut plus être réutilisé (testé).
- **Journal minimal d'anomalies** (`logSecurityEvent/getSecurityLog`, anneau borné à 20 entrées) : détecte après coup un état incohérent (ex. Premium actif sans source légitime), pour du diagnostic pendant les bêtas et après publication.
- **Refus strict en environnement 'commercial'** tant qu'aucune plateforme réelle n'est connectée : aucune fonction de paiement ou de publicité ne peut simuler un résultat positif dans ce mode.

6.2 Limitation assumée et documentée honnêtement (jamais cachée)

Un indicateur local (`localStorage`), même nommé « vérifié », reste modifiable par un joueur muni des outils de développement de son navigateur. **Aucune obfuscation côté client ne change réellement cela** — ce n'est pas un défaut de cette implémentation, c'est une limite structurelle de toute application 100% cliente. La vraie protection, une fois Play Billing connecté, sera la vérification de l'achat par la plateforme elle-même (reçu signé, `Purchase.acknowledge()`), jamais une valeur locale seule.

Scénario de contournement testé explicitement (§7) : poser directement `rupture-v0-premium=1` dans `localStorage` sans jamais passer par `purchasePremium()`. Résultat vérifié : le joueur obtient Premium (rien ne peut réellement l'en empêcher côté client), mais l'anomalie est journalisée ('premium-flag-unknown-source') car aucune source légitime n'a été enregistrée — diagnosticable, jamais silencieux.

6.3 Ce qui restera à faire une fois Play Billing réellement connecté

- Vérification du reçu d'achat côté plateforme avant d'écrire 'store-verified'.
- Restauration d'achats basée sur l'historique réel de la plateforme, pas une simple relecture locale.
- Protection contre les attributions multiples d'une même publicité à récompense, si une telle mécanique est un jour ajoutée au modèle économique (structure déjà prête, voir §4).

7. Tests réalisés et résultats de validation

7.1 Suite de tests automatisés (node:test)

86/86 tests verts (0 échec, 0 ignoré) au commit final déployé.

- Moteur de simulation, sauvegarde/migrations, UI/état — hérités des versions précédentes, tous toujours verts (aucune régression sur le cœur du jeu, non modifié cette passe).
- 11 tests dédiés à la monétisation : règle de fréquence publicitaire (première partie exemptée, joueur Premium jamais interrompu, seuil de durée exact, taux dégradé sur parties courtes, désactivable globalement), simulation vs refus en 'commercial' de l'affichage publicitaire, anti-rejeu des jetons, absence de publicité à récompense active, cycle achat/restauration Premium et son refus en 'commercial'.
- 8 tests dédiés à `save.js` : première partie complétée, source d'entitlement Premium (par défaut et explicite), journal de sécurité (contenu, ordre, anneau borné à 20 entrées, vide par défaut).
- 1 test dédié au scénario de contournement naïf du Premium local (§6.2).

7.2 Validation end-to-end (Playwright, Chromium headless)

Aucune suite Playwright n'existait avant cette passe ; une suite ciblée a été écrite spécifiquement pour valider les deux correctifs de bêta et les nouveaux parcours de monétisation sur le build réel (`npm run build + vite preview`, pas seulement du code source relu).

Vérification	Résultat
Fermeture complète + réouverture → retour au menu, jamais de reprise automatique	OK
Bouton « Reprendre la partie » restaure bien la partie après une fermeture complète	OK
Installation neuve (jamais jouée) → menu sans option de reprise	OK
Aucune erreur JS sur tout le parcours menu → tutoriel → origine → jeu	OK
Contrôles de test Premium (achat/restauration/réinitialisation) présents et fonctionnels, badge « test » visible	OK
Joueur gratuit : vitesses $\times 1/2/4$ toutes disponibles sur le build bêta réellement déployé	OK
Aucune publicité interstitielle autour de la toute première partie	OK
Ambiance sonore : horloge virtuelle sur 6 minutes → 12 événements sonores distincts, silences réels de 16 à 42s entre chaque (voir §2.2)	OK

7.3 Build et séparation dev/bêta/commercial

- `npm run build` (le build réellement déployé) : vérifié qu'il résout `env: 'beta'` dans le bundle final produit — aucune restriction Premium ne s'applique.
- `npm run build:commercial` (nouveau script, jamais utilisé pour le déploiement bêta) : vérifié qu'il résout `env: 'commercial'` dans le bundle — seul moyen aujourd'hui de tester réellement le gating $\times 4$ avant une vraie publication.
- Recherche de secrets dans le bundle produit (mots-clés clé API/mot de passe/clé privée) : aucune correspondance.

7.4 Non-régression du gameplay et de l'équilibrage

Aucun code de `src/engine/` n'a été modifié cette passe. La batterie de 702 simulations (13 stratégies $\times$ 3 difficultés $\times$ 18 origines) a été rejouée pour confirmer : joueur passif à 0% de victoires dans tous les cas, stratégie « réactive-cohérente » toujours la plus efficace (78% en normal, 39% en difficile) — profil identique à celui validé en V4.2, aux fluctuations de bruit statistique près (routes fermées par tirage aléatoire non figé, déjà caractérisées dans les rapports précédents).

7.5 Dépendances et audit de sécurité npm

0 vulnérabilité sur les dépendances de production (`npm audit --omit=dev`).

5 vulnérabilités (4 modérées, 1 haute) sur des dépendances de développement uniquement, jamais livrées dans le bundle final : (1) `esbuild/Vite` — expose le serveur de développement local, sans impact sur le site statique déployé ; (2) `uuid/xcode` — dépendance de tooling iOS de `@capacitor/cli`, jamais exécutée puisque `cap add ios` n'a jamais

été lancé. Corriger automatiquement (`npm audit fix --force`) forcerait soit une version majeure de Vite non testée, soit une version nightly de Capacitor CLI — jugé disproportionné par rapport au risque réel pour cette passe de finalisation ; à réévaluer avant une publication réelle.

7.6 Ce qui n'a PAS pu être testé ici, et pourquoi (honnêteté explicite)

Conformément à l'instruction explicite de ne jamais présenter un test non réalisable comme réussi :

- **Achat Premium et publicité réels** : aucun compte Google Play Billing ni compte AdMob n'existe. Seul le refus d'agir en environnement 'commercial' a pu être vérifié — pas un vrai paiement.
- **Projet Android natif** : `npx cap add android` n'a pas été exécuté (nécessite Android Studio/SDK, absents de cet environnement distant). Voir §9.
- **« Agrément » subjectif de la musique** : la nature réellement plus agréable de la nouvelle ambiance ne peut être confirmée qu'en bêta manuelle humaine, comme le défaut initial l'a été. Ce rapport vérifie la structure objective (pas de son continu, silences réels mesurés) — pas le ressenti subjectif final, qui reste à confirmer par vous.

8. Architecture finale

Aucun changement structurel majeur cette passe au-delà de l'ajout des services de monétisation déjà préparés depuis docs/ARCHITECTURE.md (interfaces de service anticipées avant même qu'un besoin réel ne les impose).

Dossier / fichier	Rôle
src/engine/	Noyau pur (territoires, équilibrage, état, simulation, migrations) — aucune connaissance du DOM ni de localStorage. Non modifié cette passe.
src/ui/	Templating à partir de l'état reçu (carte, HUD, écrans). Étendu cette passe : section Premium dans les paramètres, écran interstitiel de test.
src/save.js	Seul point de contact localStorage. Étendu cette passe : première partie complétée, source d'entitlement Premium, journal de sécurité.
src/services/	ads.js et premium.js réellement implémentés cette passe (auparavant des adaptateurs no-op). analytics.js inchangé (toujours no-op, analyticsEnabled=false).
src/config/runtime.js	Point de vérité unique pour l'environnement (dev/bêta/commercial) et les paramètres commerciaux. BUILD_ENV rendu réellement configurable via VITE_BUILD_ENV.
src/main.js	Seule « colle » DOM/moteur/sauvegarde/services. Correctif de persistance et câblage de l'évaluation publicitaire ajoutés ici.
src/audio/audio.js	Moteur musical entièrement réécrit (ambiance événementielle). Effets sonores inchangés.
capacitor.config.json	Nouveau. Configuration de packaging Android/iOS (voir §9).
.env.example / .gitignore	Nouveaux/étendus. Aucune valeur réelle committée (voir §10).

9. Passage de RUPTURE à une publication réelle

Cette section est pensée comme une feuille de route à utiliser le jour où vous déciderez réellement de publier RUPTURE. Rien ici ne prétend qu'une intégration réelle existe alors qu'elle ne l'est pas.

9.1 Déjà prêt (vérifié, réel)

- Stratégie de packaging actée : Capacitor (encapsule le build web existant dans une coquille native — aucune réécriture du moteur ni de l'UI), déjà recommandée dans docs/TRAJECTOIRE_COMMERCIALE.md avant cette passe.
- @capacitor/core, @capacitor/android (dépendances) et @capacitor/cli (dev) installés et compatibles avec le build Vite existant — vérifié (npm install propre, aucun conflit).
- capacitor.config.json créé (webDir: "dist" — le build existant est embarqué tel quel).
- Séparation dev/bêta/commercial réellement fonctionnelle et testée localement (npm run build:commercial).
- Icônes PWA existantes (public/icons/) réutilisables comme base pour les icônes Android.
- Aucune permission Android sensible nécessaire aujourd'hui (ni caméra, ni localisation, ni contacts) — seul l'accès réseau serait requis une fois un SDK publicitaire/analytics réel connecté, ajouté automatiquement par Capacitor.

9.2 Ne fonctionne qu'en environnement de test aujourd'hui

- Achat Premium et affichage publicitaire : code réel et testé (\$4/\$5), mais aucune plateforme n'est connectée — résultat marqué « simulated » partout où l'interface l'affiche.
- Restriction de vitesse x4 Premium : le mécanisme fonctionne (testé, §7.3) mais reste inactif tant que le build déployé n'est pas construit avec VITE_BUILD_ENV=commercial.

9.3 Reste à connecter avant publication

- **Compte développeur Google Play** (frais unique, ~25 \$US) — à créer par vous.
- **Compte AdMob** + création d'une application AdMob (fournit l'App ID à placer dans .env.local, jamais committé).
- **Compte marchand Google Play** (pour percevoir les revenus Premium) — se configure lors de la création du produit dans la Play Console.
- **Plugin Capacitor AdMob** à intégrer dans src/services/ads.js uniquement (aucun autre fichier à toucher) : remplace le corps de showInterstitialAd(). shouldShowInterstitial() n'a besoin d'aucune modification.
- **Plugin Capacitor Play Billing** à intégrer dans src/services/premium.js uniquement : remplace le corps de purchasePremium()/restorePurchases(), et doit écrire 'store-verified' (jamais 'simulated-test') comme source d'entitlement une fois le reçu vérifié par la plateforme.
- **Consentement publicitaire RGPD/EEE** : SDK gratuit Google User Messaging Platform (UMP), à intégrer avant le premier appel publicitaire réel si l'audience européenne est ciblée.

9.4 Étapes restantes — Android / Google Play

- npm install puis npm run build (génère dist/).
- npx cap add android — génère le projet natif Gradle complet (exclu du dépôt, voir .gitignore). **Non exécuté dans cette passe** : nécessite Android SDK/Studio, absents de cet environnement d'exécution distant. L'exécuter à l'aveugle aurait produit un projet natif jamais réellement vérifié.
- npx cap sync android après chaque modification de dist/ ou du config.
- Ouvrir android/ dans Android Studio : compiler, tester sur émulateur/appareil réel, générer un App Bundle (.aab) signé.
- Confirmer/modifier l'appId (com.scuillerjackoss.rupture est un placeholder) AVANT toute première soumission — permanent une fois publié.
- Générer les icônes adaptatives Android complètes (l'outil @capacitor/assets, à installer séparément, génère toutes les résolutions depuis l'icône source existante).

- Synchroniser manuellement `versionCode/versionName` (`android/app/build.gradle`) avec `package.json` à chaque publication — Capacitor ne le fait pas automatiquement.
- Rédiger la politique de confidentialité (hébergement statique gratuit suffisant), le formulaire « Sécurité des données », le questionnaire de classification par âge, la fiche store (description, captures d'écran) — décisions produit, non traitées par cette passe technique.

9.5 iOS / App Store (si un jour envisagé)

Explicitement hors périmètre de l'audit économique retenu (Google Play/Android uniquement) et non traité en détail par cette passe. Nécessiterait : un compte Apple Developer Program (payant, annuel), `npx cap add ios` (jamais exécuté), Xcode (macOS requis, absent de tout environnement utilisé jusqu'ici), un processus de signature et de soumission App Store Connect distinct de celui de Google Play.

9.6 Stratégies de vérification (achat / publicité à récompense)

- **Achat Premium** : une fois Play Billing connecté, ne jamais faire confiance à une réponse client seule — vérifier le reçu signé via l'API Play Developer (ou déléguer à `Purchase.acknowledge()` côté plugin natif) avant d'écrire 'store-verified'. La structure actuelle (source d'entitlement séparée) est prête à recevoir cette vérification sans réécriture ailleurs.
- **Publicité à récompense** : aucune mécanique n'existe dans le modèle retenu aujourd'hui. Si une est ajoutée plus tard, la structure de jetons anti-rejeu à usage unique (\$4) est déjà prête à empêcher une double attribution — mais nécessitera aussi la confirmation de complétion réelle fournie par le SDK publicitaire natif (callback "reward earned"), pas uniquement le jeton local.

9.7 Besoin éventuel d'un petit backend

Non nécessaire pour publier RUPTURE tel que préparé par cette passe : la vérification d'achat Play Billing peut se faire entièrement via le SDK natif côté client pour un lancement initial. Un petit backend (fonctions serverless) ne deviendrait utile que pour une vérification serveur-à-serveur plus robuste contre la falsification avancée — à envisager seulement si des abus réels sont constatés après publication, jamais construit par anticipation. Règle financière durable si un jour ajouté : toujours configurer des alertes de budget et des plafonds de dépense stricts AVANT toute intégration à facturation variable non plafonnée.

10. Secrets, identifiants et ce que vous devrez créer vous-même

Rien de ce qui suit n'a été créé, deviné ou simulé par cette passe — voir docs/SECRETS_ET_PRODUCTION.md pour le détail complet.

Secret / identifiant	Où il apparaît	Comment le protéger
Mot de passe du keystore Android	Génération locale (keytool), Play App Signing	Gestionnaire de mots de passe personnel. Une perte = impossible de publier une mise à jour sous le même appld.
Fichier keystore (.jks/.keystore)	Généré localement	Sauvegarde chiffrée hors dépôt (déjà exclu par .gitignore). Play App Signing recommandé — Google conserve la clé d'upload.
Clé de compte de service Google Play	Play Console > API Access, si publication automatisée future	Jamais dans le dépôt ; en secret CI GitHub Actions si un pipeline de publication est ajouté un jour — aucun n'existe aujourd'hui.
google-services.json (si Firebase ajouté)	Firebase Console	Déjà dans .gitignore. Identifie le projet Firebase, pas un mot de passe en soi, mais à ne pas exposer par prudence.
App ID AdMob	Compte AdMob	Pas un secret sensible en soi (peut apparaître dans l'APK public) mais spécifique à votre compte — à placer dans .env.local (jamais committé).

Vérifié à ce commit : aucun fichier .env (hors .env.example, sans valeur réelle) n'est suivi par git ; recherche de motifs de secrets (clés API, mots de passe, clés privées) sur le code source et sur le bundle produit final : aucune correspondance.

11. Limites connues, délibérément conservées

- **Résilience face à la Réponse mondiale** (remontée en bêta V4.1) : quatre mécanismes de renforcement ont été testés rigoureusement en V4.2 (90 parties par valeur), tous produisant soit aucun effet mesurable soit une falaise catastrophique de taux de victoire. Aucun changement n'a été forcé, conformément à la consigne explicite reçue à l'époque. Cette passe V5 n'a pas rouvert cette investigation (hors périmètre d'une passe de finalisation).
- **Aucune mécanique de publicité à récompense** : le modèle économique validé n'en définit aucune. Une structure anti-rejeu existe pour en accueillir une plus tard sans réécriture, mais rien n'a été inventé pour combler cette absence.
- **iOS non traité** : hors périmètre de l'audit économique retenu (Android/Google Play uniquement).
- **Sécurité du Premium local** : limitation structurelle assumée de toute application 100% cliente tant que Play Billing n'est pas connecté (voir §6.2) — jamais présentée comme une vraie protection.
- **Icône adaptative Android** : l'icône maskable PWA existante fonctionnera telle quelle, mais une séparation premier-plan/arrière-plan plus aboutie nécessiterait une retouche graphique non produite ici (aucun résultat visuel deviné).

12. Verdict final

Définition de « terminé » pour cette mission de finalisation — vérifiée point par point :

Critère (demande de finalisation)	État
Les deux défauts remontés en bêta sont corrigés	OK — Fait, vérifié (§2, §7)
Le modèle économique déjà validé est correctement intégré	OK — Fait, sans réinvention (§3-§5)
L'architecture de monétisation est raisonnablement sécurisée	OK — Fait, limites assumées (§6)
Le projet est préparé pour un futur packaging mobile	OK — Préparé, non exécuté faute d'outils (§9)
Aucune régression majeure détectée	OK — 86/86 tests, 702 simulations, E2E complet (§7)
La version est déployée sur l'URL permanente	OK — commit bc3b863, vérifié via l'API GitHub Actions
Le rapport final est produit	OK — Ce document
Les éléments nécessitant vos futurs comptes/services sont identifiés clairement	OK — §9, §10

RUPTURE V5 conclut le premier cycle complet — concept → prototype → développement → équilibrage → bêta réelle → monétisation → sécurisation → préparation stores → documentation — sans avoir cherché une perfection artificielle.

Enseignements à retenir pour les prochains jeux : distinguer explicitement RÉEL/SIMULÉ/PRÉPARÉ dès la conception des services (Premium, publicité) évite toute ambiguïté ultérieure sur ce qui est réellement prêt pour une publication ; une correction de bug de bêta réelle sur un défaut perceptible (audio) mérite une vérification structurelle automatisée (horloge virtuelle) plutôt qu'une simple relecture de code ; et accepter explicitement qu'une investigation de gameplay rigoureuse puisse conclure « aucun changement sûr trouvé » sans qu'il s'agisse d'un échec du processus.